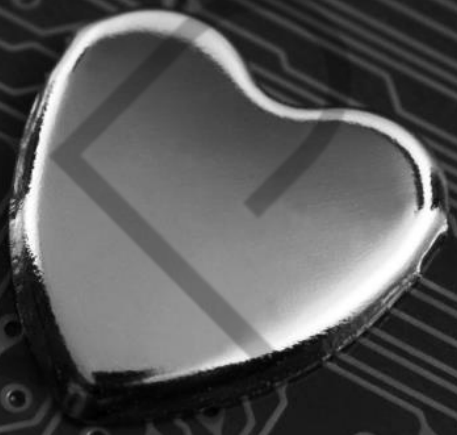
FIAP ON



05

CARDIOIA EM AÇÃO: CONSTRUINDO SAÚDE INTELIGENTE COM REACTJS + VITE

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Interface gráfica Terminal do usuário....8
Figura 2 - Interface gráfica VsCode....9
Figura 3 - Interface gráfica VsCode – Terminal....10
Figura 4 - Estrutura de pastas recomendada....11
Figura 5 - Interface gráfica VsCode – Diretórios e Arquivos....12
Figura 6 - FormularioPaciente.jsx....14
Figura 7 - ListaPacientes.jsx....21
Figura 8 - Caminho do arquivo “src/App.jsx”....24
Figura 9 - Interface do Projeto no Navegador sem estilização de CSS....30
Figura 10 - Interface gráfica VsCode – Diretórios e Arquivos....31
Figura 11 - Interface gráfica VsCode – Import do css....38
Figura 12 - Interface do Projeto no Navegador com estilização de CSS....38
Figura 13 - Aplicação inicial sem dados preenchidos....42
Figura 14 - Aplicação inicial com os dados preenchidos....43
Figura 15 - Aplicação com novo dado preenchido....44

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Implementação do FormularioPaciente.jsx 16
Código-fonte 2 - Função FormularioPaciente 17
Código-fonte 3 - Função de envio do formulário 18
Código-fonte 4 - Estrutura visual do formulário 19
Código-fonte 5 - Função para listar pacientes 21
Código-fonte 6 - Exportar a lista de pacientes 22
Código-fonte 7 - Retornar lista não ordenada 22
Código-fonte 8 - Renderização de lista de pacientes com a função map 22
Código-fonte 9 - Exibição individual dos dados de cada paciente na lista 23
Código-fonte 10 - Componente raiz com formulário, lista e persistência local 25
Código-fonte 11 - Importação de hooks, componentes e estilos 26
Código-fonte 12 - Componente raiz com estados de pacientes e carregamento 26
Código-fonte 13 - Carregamento inicial dos pacientes via localStorage 27
Código-fonte 14 - Salvando pacientes no localStorage após carregamento 27
Código-fonte 15 - Função para incluir novo paciente na lista 28
Código-fonte 16 - Estrutura da interface com título, formulário e lista de pacientes 28
Código-fonte 17 - Estilos globais, formulário e lista de pacientes em CSS 33
Código-fonte 18 - Configuração visual básica da página (body) 33
Código-fonte 19 - Estilo do título principal da aplicação 33
Código-fonte 20 - Layout e aparência do formulário principal 34
Código-fonte 21 - Estilo dos campos de entrada do formulário 34
Código-fonte 22 - Estilo do botão do formulário com efeito hover 35
Código-fonte 23 - Estilo da lista de pacientes e de cada item 35
Código-fonte 24 - Layout em linha para os campos de pressão arterial 36
Código-fonte 25 - Ajuste dos inputs dentro do bloco de pressão arterial 36
Código-fonte 26 - Estilo do separador entre os campos de pressão arterial 36
Código-fonte 27 - Estilo do texto de exemplo para preenchimento da pressão arterial 37
Código-fonte 28 - Atualização automática do localStorage ao mudar a lista de pacientes 39
Código-fonte 29 - Carregamento inicial dos pacientes salvos no localStorage 40

TRIAL

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Criação do projeto React com Vite no diretório cardioia-app 8
Comando de prompt 2 - Acessa o diretório do projeto recém-criado 8
Comando de prompt 3 - Instalar dependências do projeto.....9
Comando de prompt 4 - Comando para instalar dependências do projeto 10
Comando de prompt 5 - Criando diretórios no projeto 11
Comando de prompt 6 - Import do React.....16
Comando de prompt 7 - Registro e envio das alterações para o repositório Git 46

FIAP

SUMÁRIO

1 CARDIOIA EM AÇÃO: CONSTRUINDO SAÚDE INTELIGENTE COM REACTJS + VITE 6
1.1 Estrutura inicial do projeto 6
1.2 CardioIA: Quando a tecnologia pulsa com a vida....6
2 CRIAÇÃO DO PROJETO CARDIOIA 8
3 BATIMENTOS DE DADOS 14
4 COMPONENTE LISTAPACIENTES 21
5 INTEGRAÇÃO NO APP.JSX – ORGANIZANDO O FLUXO COMPLETO DA FASE 1 24
6 APLICANDO ESTILO À INTERFACE – PRIMEIRA CAMADA DE APRESENTAÇÃO....30
6.1 Onde criar o arquivo de estilo....31
6.1.1 Explicação por Blocos do styles.css....33
7 TESTANDO O FUNCIONAMENTO E VISUALIZANDO OS DADOS NO NAVEGADOR 39
7.1 O que é o localStorage?....39
7.2 Onde está o código que grava no localStorage?....39
7.3 Como visualizar os dados salvos no localstorage do navegador? 40
7.4 Por que estamos usando localStorage?....41
7.5 Conclusão didática 41
8 VISUALIZANDO O COMPORTAMENTO DO LOCALSTORAGE EM TEMPO REAL 42
9 VERSIONANDO O PROJETO CARDIOIA NO GITHUB 45
9.1 Etapas para versionar o projeto no GitHub 45
10 FINALIZANDO A FASE 1 – CADASTRO E PERSISTÊNCIA DE PACIENTES ...47
REFERÊNCIAS....48

FIAP

1 CARDIOIA EM AÇÃO: CONSTRUINDO SAÚDE INTELIGENTE COM REACTJS + VITE

1.1 Estrutura inicial do projeto

Antes de iniciarmos, vamos relembrar o que você já aprendeu:

- **Capítulo 1:** aprendemos os fundamentos do ReactJS, props, estado, ciclos de vida, estilização e gerenciamento de eventos.
- **Capítulo 2:** vimos Hooks avançados (useContext, useReducer), Context API, integração com APIs de IA e autenticação JWT.

Agora, criaremos a base do projeto que suportará as próximas fases.

1.2 CardioIA: Quando a tecnologia pulsa com a vida

Imagine um futuro — que já começa agora — onde os sistemas inteligentes auxiliam médicos a prever riscos cardíacos antes mesmo que os primeiros sintomas se manifestem. Sensores vestíveis (*wearables*), como relógios inteligentes ou faixas biomédicas, captam em tempo real batimentos cardíacos, pressão arterial e informações clínicas importantes. Esses dados, que antes ficariam esquecidos em papéis ou planilhas, agora são processados por algoritmos que detectam padrões invisíveis ao olhar humano.

Interfaces web intuitivas transformam essa inteligência em diagnósticos visuais, acessíveis e compreensíveis até para quem não tem formação médica. E o mais incrível? Você vai começar a construir isso agora.

Neste capítulo, você iniciará a jornada prática do **CardioIA**, uma aplicação web criada com **ReactJS + Vite** que simula um sistema de monitoramento e apoio à saúde cardiovascular. A proposta nasce da combinação entre tecnologia, Inteligência Artificial e empatia, integrando o que aprendemos até aqui com desafios reais da área da saúde.

Com base no que você já domina em React e nos novos aprendizados que virão, você será capaz de:

- Montar uma estrutura de projeto profissional com diretórios bem organizados;
- Criar componentes reutilizáveis com foco em usabilidade;
- Simular a coleta e a análise de dados clínicos relevantes;
- Trabalhar com conceitos reais de sistemas inteligentes aplicados à saúde;
- Desenvolver interfaces que tenham propósito social e impacto direto na vida das pessoas.

Mais do que uma aula, este capítulo é um convite para aplicar tecnologia de maneira significativa. Ao final do módulo, seu projeto poderá servir como base para ideias futuras em inovação médica, hackathons, TCCs ou até startups.

Acompanhe com atenção. Pratique com dedicação. E lembre-se: cada linha de código que você escrever aqui pode, no futuro, ajudar a salvar vidas.

2 CRIAÇÃO DO PROJETO CARDIOIA

Antes de iniciar, abra o terminal do seu sistema e crie uma pasta próxima à raiz do disco para evitar erros com caminhos longos. Recomendamos usar diretórios como: **C:\projetos-react** ou **C:\dev**.

Dentro dessa pasta, será necessário criar o projeto com React + Vite.

Passo 1 – Criar o projeto com Vite

No terminal, digite o seguinte comando para iniciar um novo projeto React com Vite:

```
npm create vite@latest cardioia-app -- --template react
```

Comando de prompt 1 - Criação do projeto React com Vite no diretório cardioia-app
Fonte: Elaborado pelo autor (2025)

Durante a execução, o terminal pedirá confirmação de nome e template. Confirme os padrões sugeridos.



Figura 1 - Interface gráfica Terminal do usuário
Fonte: Elaborado pelo autor (2025)

Após a criação do projeto, entre na pasta do projeto com:

```
cd cardioia-app
```

Comando de prompt 2 - Acessa o diretório do projeto recém-criado
Fonte: Elaborado pelo autor (2025)

Em seguida, instale as dependências padrões do projeto com:

```
npm install
```

Comando de prompt 3 - Instalar dependências do projeto
Fonte: Elaborado pelo autor (2025)

Após terminar a instalação do seu projeto base inicial, abra seu VsCode no diretório raiz do projeto, que deverá ter a seguinte interface:

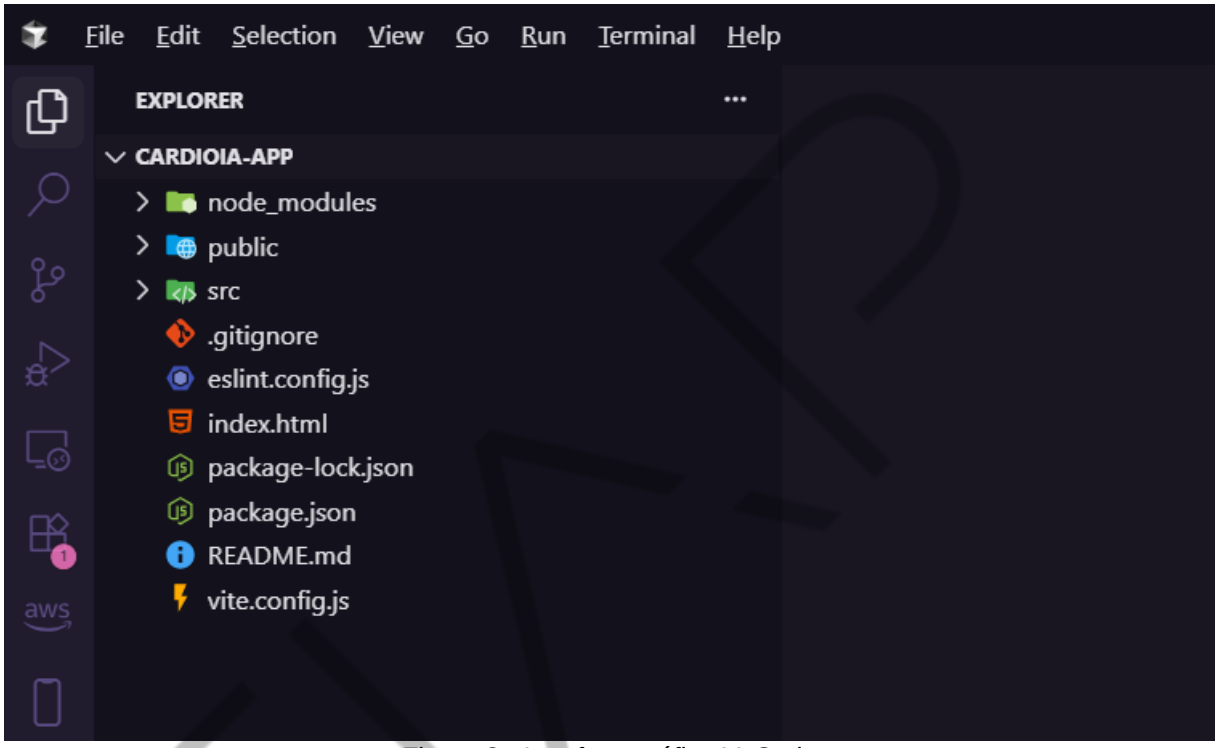


Figura 2 - Interface gráfica VsCode
Fonte: Elaborado pelo autor (2025)

Antes de começarmos a construir as funcionalidades mais importantes do projeto **CardioIA**, precisamos preparar o ambiente com algumas bibliotecas que tornarão o desenvolvimento mais prático, organizado e visualmente interessante. Essas bibliotecas complementam o React com recursos essenciais como requisições HTTP, navegação entre páginas e gráficos interativos.

Passo 2 – Instalação de dependências adicionais

Com seu projeto já criado e aberto no Visual Studio Code, instalaremos as bibliotecas que serão utilizadas nas próximas fases.

Primeiramente, abra o terminal do VS Code usando o atalho CTRL + J ou clicando em “Terminal” → “Novo Terminal” no menu superior.

Em seguida, execute o comando a seguir no terminal para instalar os pacotes necessários:

```
npm install axios react-router-dom recharts
```

Comando de prompt 4 - Comando para instalar dependências do projeto
Fonte: Elaborado pelo autor (2025)

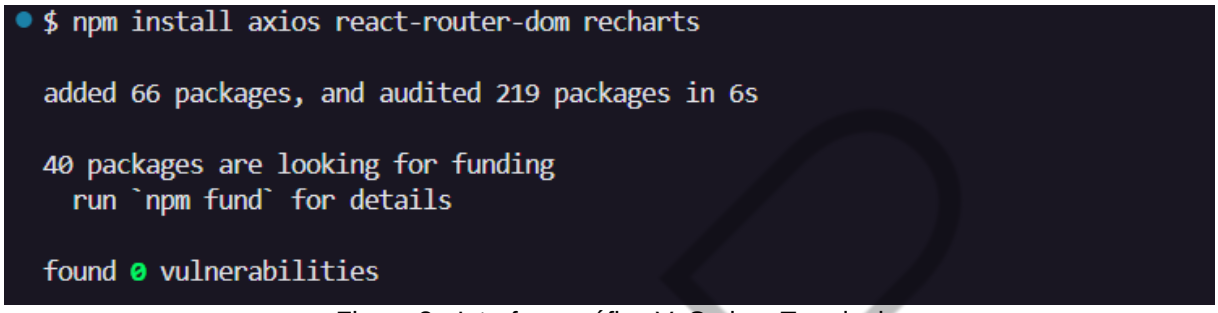


Figura 3 - Interface gráfica VsCode – Terminal
Fonte: Elaborado pelo autor (2025)

Esses pacotes têm funções bem específicas e úteis no nosso projeto:

- **axios:** facilita a comunicação com APIs externas através de requisições HTTP (GET, POST, PUT, DELETE, entre outros), o que é essencial quando simulamos integrações com sistemas de IA ou bancos de dados.
- **react-router-dom:** permite criar múltiplas páginas dentro da aplicação React sem recarregar a tela inteira, característica típica de aplicações do tipo SPA (Single Page Application).
- **recharts:** biblioteca de gráficos compatível com React, ideal para representar visualmente dados médicos, como batimentos cardíacos, níveis de pressão e outros indicadores vitais.

Essas dependências serão utilizadas em várias fases do CardioIA, especialmente quando começarmos a exibir gráficos dinâmicos e criar rotas entre as telas do sistema.

Passo 3 – Estrutura de pastas

Agora que o projeto **CardioIA** foi criado e está com as dependências instaladas, é hora de organizar a estrutura da pasta ***src***. Manter um projeto bem estruturado desde o início é fundamental para garantir legibilidade, escalabilidade e facilidade de manutenção, principalmente em aplicações que vão crescer ao longo do tempo — como é o caso do CardioIA.

Como criar os diretórios:

Você pode criar os diretórios de duas formas:

1. Utilizando o terminal com o comando mkdir: no terminal do VS Code (atalho Ctrl + J), execute os seguintes comandos, um por um:

```
cd src
mkdir components
mkdir pages
mkdir contexts
mkdir hooks
mkdir services
```

Comando de prompt 5 - Criando diretórios no projeto
Fonte: Elaborado pelo autor (2025)

Observação: a pasta ***assets*** já é criada automaticamente pelo Vite e pode ser aproveitada para armazenar imagens e outros arquivos estáticos.

2. Criando diretamente pelo VS Code: se preferir criar visualmente, clique com o botão direito na pasta **src** no painel lateral esquerdo do VS Code.

   Escolha a opção **"New Folder"** (Nova Pasta).

   Digite o nome da pasta e pressione Enter.

   Repita esse processo para cada pasta listada abaixo.

Estrutura de pastas recomendada:

Após a criação, sua estrutura dentro da pasta src deverá se parecer com esta:

```
src
├ components   // Componentes reutilizáveis da interface (formulários, botões, etc...)
├ pages        // Telas completas da aplicação (Home, Dashboard, Cadastro, etc.)
├ contexts     // Contextos globais usando Context API para compartilhamento de estado
├ hooks        // Hooks personalizados reutilizáveis
├ assets       // Imagens, ícones, fontes e arquivos (gerada automaticamente pelo Vite)
└ services     // Arquivos responsáveis por chamadas a APIs e regras de negócio
```

Figura 4 - Estrutura de pastas recomendada
Fonte: Elaborado pelo autor (2025)

Essa organização será utilizada durante todas as fases do desenvolvimento. Sempre que novos arquivos forem criados, indicaremos em qual pasta eles devem ser salvos.

A estrutura do projeto do seu VsCode deverá ficar semelhante a esta imagem:

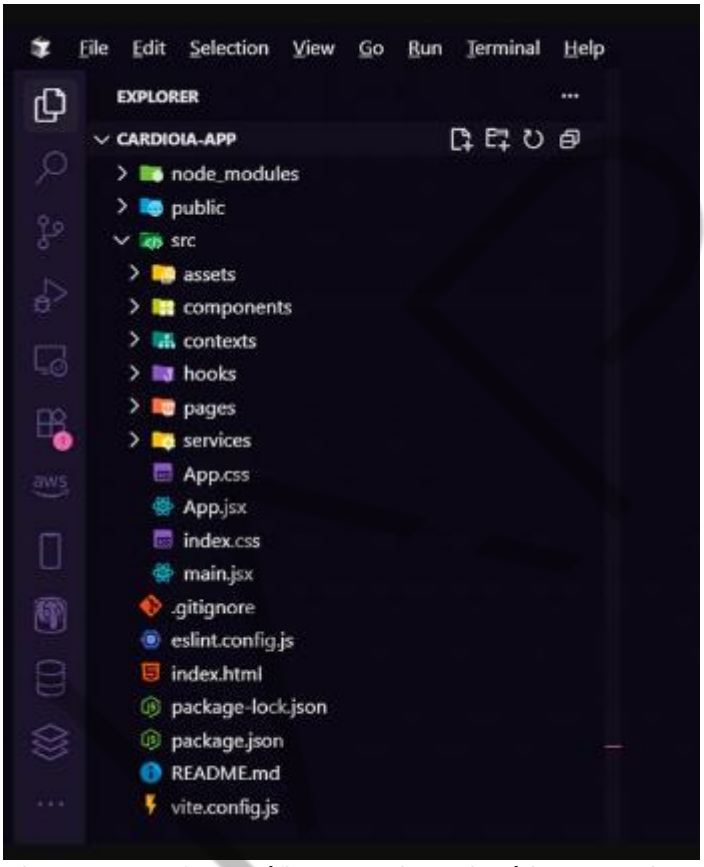


Figura 5 - Interface gráfica VsCode – Diretórios e Arquivos
Fonte: Elaborado pelo autor (2025)

Observação sobre ícones no VS Code:

Se os diretórios e arquivos no seu VS Code estiverem com ícones diferentes dos mostrados nesta apostila (por exemplo, pastas com ícones coloridos ou personalizados como na imagem do professor), isso se deve à instalação de uma extensão chamada **Material Icon Theme**.

Caso queira personalizar seu Visual Studio Code com os mesmos ícones visuais para pastas e arquivos, siga os passos:

- No VS Code, clique no menu lateral esquerdo em Extensões (ícone de quadrado).
- Pesquise por Material Icon Theme.

- Clique em Instalar.
- Após a instalação, uma mensagem aparecerá perguntando se você deseja ativar o tema de ícones. Clique em Yes.

Se não quiser instalar, sem problemas! Você pode seguir normalmente com o desenvolvimento do projeto. Os ícones dos diretórios e arquivos não afetam o funcionamento do código. Use a estrutura de pastas conforme explicada anteriormente, mesmo que a aparência visual seja diferente da demonstrada.

FIAP

3 BATIMENTOS DE DADOS

Imagine que você é responsável pela triagem inicial de pacientes em uma startup de saúde inteligente. Nessa etapa, precisamos garantir que os dados clínicos básicos de cada paciente sejam registrados de forma organizada, limpa e segura. Esta é a base para qualquer sistema inteligente de apoio à saúde: dados confiáveis e bem estruturados.

Nesta primeira fase do CardioIA, você irá criar um formulário funcional para cadastrar pacientes cardiológicos simulados. Esses dados serão exibidos em tempo real e armazenados localmente no navegador para simular persistência.

Objetivos da fase:

- Criar componentes React para formulários reutilizáveis.
- Utilizar useState para controlar o estado dos campos do formulário.
- Validar e limpar os dados antes do envio.
- Persistir os dados com localStorage como se fosse um banco local.

Onde criar:

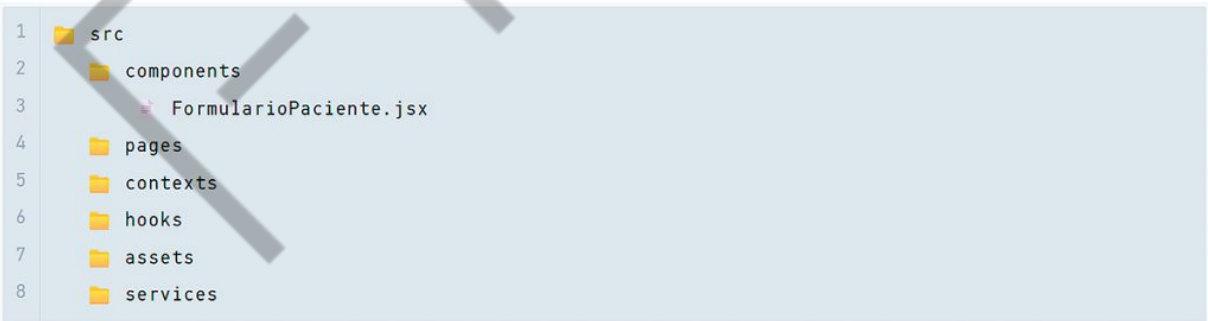


Figura 6 - FormularioPaciente.jsx
Fonte: Elaborado pelo autor (2025)

O que será feito: neste componente, você vai criar um formulário com campos de entrada para nome, idade e pressão arterial do paciente. Sempre que o formulário for enviado, os dados serão validados e enviados para a lista principal da aplicação.

```
     import { useState } from "react";

     export function FormularioPaciente({ onAddPaciente }) {
        const [nome, setNome] = useState("");
        const [idade, setIdade] = useState("");
        const    [pressaoSistolica,    setPressaoSistolica]    =
useState("");
        const   [pressaoDiastolica,   setPressaoDiastolica]   =
useState("");

        function handleSubmit(e) {
           e.preventDefault();
           if   (!nome   ||   !idade   ||   !pressaoSistolica   ||
!pressaoDiastolica) return;

           const paciente = {
              nome: nome.trim(),
              idade: parseInt(idade),
              pressaoSistolica: parseInt(pressaoSistolica),
              pressaoDiastolica: parseInt(pressaoDiastolica),
           };

           onAddPaciente(paciente);

           // Limpa os campos após o cadastro
           setNome("");
           setIdade("");
           setPressaoSistolica("");
           setPressaoDiastolica("");
        }

        return (
           <form onSubmit={handleSubmit}>
              <input
                 value={nome}
                 onChange={(e) => setNome(e.target.value)}
                 placeholder="Nome"
              />
              <input
                 value={idade}
                 onChange={(e) => setIdade(e.target.value)}
                 placeholder="Idade"
                 type="number"
                 min="1"
                 max="120"
              />

              <div className="pressao-campos">
                 <input
                    value={pressaoSistolica}
```

```
                    onChange={(e)                                    =>
setPressaoSistolica(e.target.value)}
                    placeholder="Pressão Sistólica (máxima)"
                    type="number"
                    min="60"
                    max="250"
                    required
                />
                <span className="separador-pressao">x</span>
                <input
                    value={pressaoDiastolica}
                    onChange={(e)                                    =>
setPressaoDiastolica(e.target.value)}
                    placeholder="Pressão Diastólica (mínima)"
                    type="number"
                    min="40"
                    max="150"
                    required
                />
            </div>

            <p  className="exemplo-pressao">Exemplo:  120  x  80
mmHg</p>

            <button      type="submit">Cadastrar      Paciente
</button>
        </form>
    );
}
```

Código-fonte 1 - Implementação do FormularioPaciente.jsx
Fonte: Elaborado pelo autor (2025)

Análise do componente FormularioPaciente.jsx: veja o que cada parte do código acima faz, explicada passo a passo para facilitar o entendimento e a prática.

Importação do Hook:

Importa o useState, hook nativo do React usado para criar e atualizar variáveis de estado no componente funcional.

```
import { useState } from "react";
```

Comando de prompt 6 - Import do React
Fonte: Elaborado pelo autor (2025)

Declaração do componente e dos estados:

Define o componente FormularioPaciente, que recebe uma função onAddPaciente por props. Os quatro estados armazenam os dados digitados pelo usuário: nome, idade, pressão sistólica e pressão diastólica.

```
export function FormularioPaciente({ onAddPaciente }) {
   const [nome, setNome] = useState("");
   const [idade, setIdade] = useState("");
   const [pressaoSistolica, setPressaoSistolica] =
useState("");
   const [pressaoDiastolica, setPressaoDiastolica] =
useState("");
```

Código-fonte 2 - Função FormularioPaciente
Fonte: Elaborado pelo autor (2025)

Função de envio do formulário:

- Impede o comportamento padrão do formulário (recarregar a página).
- Valida se todos os campos foram preenchidos.
- Cria um objeto paciente com os dados tipados corretamente.
- Envia o paciente para o componente pai via onAddPaciente.
- Limpa os campos após o envio.

```
function handleSubmit(e) {
   e.preventDefault();
   if (!nome || !idade || !pressaoSistolica ||
!pressaoDiastolica) return;

   const paciente = {
      nome: nome.trim(),
      idade: parseInt(idade),
      pressaoSistolica: parseInt(pressaoSistolica),
      pressaoDiastolica: parseInt(pressaoDiastolica),
   };

   onAddPaciente(paciente);

   // Limpa os campos após o cadastro
   setNome("");
   setIdade("");
   setPressaoSistolica("");
   setPressaoDiastolica("");
}
```

Código-fonte 3 - Função de envio do formulário
Fonte: Elaborado pelo autor (2025)

Estrutura visual do formulário:

- O formulário possui inputs controlados para nome, idade, pressão sistólica e diastólica.
- A área de pressão é organizada com uma com classe pressao-campos, facilitando a estilização visual lado a lado.
- O span com o x estilizado separa os dois campos de forma clara.
- Abaixo, uma dica visual exibe o formato esperado (exemplo: 120 x 80 mmHg).
- O botão final envia o formulário.

```
     <form onSubmit={handleSubmit}>
        <input
           value={nome}
           onChange={(e) => setNome(e.target.value)}
           placeholder="Nome"
        />
        <input
           value={idade}
           onChange={(e) => setIdade(e.target.value)}
           placeholder="Idade"
           type="number"
           min="1"
           max="120"
        />
        <div className="pressao-campos">
           <input
              value={pressaoSistolica}
              onChange={(e)                                    =>
setPressaoSistolica(e.target.value)}
              placeholder="Pressão Sistólica (máxima)"
              type="number"
              min="60"
              max="250"
              required
           />
           <span className="separador-pressao">x</span>
           <input
              value={pressaoDiastolica}
              onChange={(e)                                    =>
setPressaoDiastolica(e.target.value)}
              placeholder="Pressão Diastólica (mínima)"
              type="number"
              min="40"
              max="150"
              required
           />
        </div>
        <p   className="exemplo-pressao">Exemplo:   120   x   80
mmHg</p>

        <button type="submit">Cadastrar Paciente</button>
     </form>
```

Código-fonte 4 - Estrutura visual do formulário
Fonte: Elaborado pelo autor (2025)

Conclusão didática:

- O formulário está completamente controlado via React.

- Agora, os dados de pressão são armazenados separadamente, permitindo análises mais detalhadas em fases posteriores do projeto.
- O componente mantém o princípio da separação de responsabilidades: coleta os dados e delega o armazenamento ao componente pai via props.
- Esse modelo permite evolução para análises com IA, validações avançadas e integração com banco de dados ou APIs.



4 COMPONENTE LISTAPACIENTES

O componente **ListaPacientes** é responsável por exibir dinamicamente na tela todos os pacientes cadastrados no formulário anterior. Ele recebe uma lista (array) de objetos chamada pacientes e percorre cada item com o método .map(), imprimindo as informações de nome, idade e pressão arterial.

Esse componente não armazena estado próprio, apenas renderiza as informações recebidas como propriedade (**props**). Isso garante a separação clara de responsabilidades entre o componente que coleta dados (**FormularioPaciente**) e o que os exibe (**ListaPacientes**).

Função didática: esse componente ensina como renderizar listas dinâmicas em React, utilizando a prop key para garantir desempenho e evitar erros no DOM virtual.

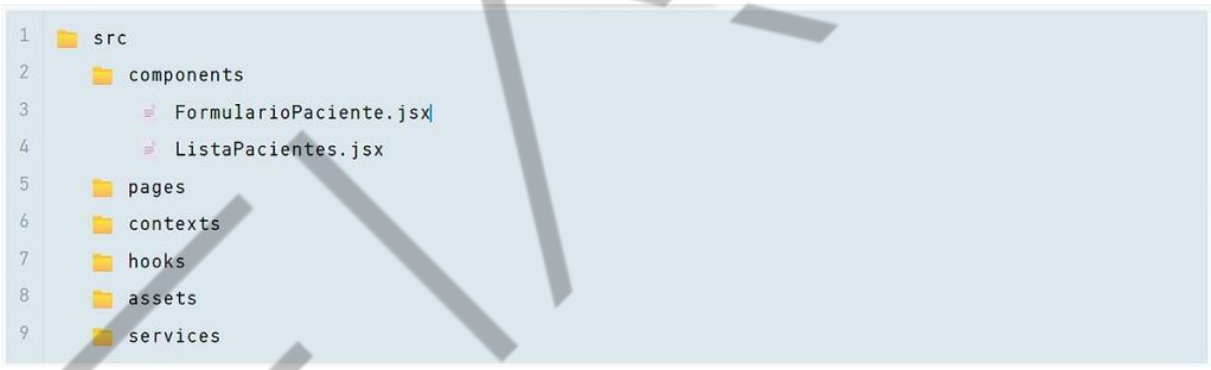


Figura 7 - ListaPacientes.jsx
Fonte: Elaborado pelo autor (2025)

```
export function ListaPacientes({ pacientes }) {
   return (
      <ul>
         {pacientes.map((p, i) => (
            <li key={i}>{p.nome} - {p.idade} anos -
{p.pressaoSistolica}x{p.pressaoDiastolica} mmHg</li>
         ))}
      </ul>
   );
}
```

Código-fonte 5 - Função para listar pacientes
Fonte: Elaborado pelo autor (2025)

Análise Comentada do Código – ListaPacientes.jsx:

Esse componente é responsável por exibir dinamicamente todos os pacientes cadastrados, organizando cada registro em uma lista. A renderização é baseada na lista pacientes, que vem como propriedade (prop) do componente pai **App.jsx**.

- **Declaração do componente funcional:**
  - Define um componente chamado ListaPacientes, que é exportado para ser utilizado em outros arquivos.
  - Recebe como parâmetro uma propriedade chamada **pacientes**, que é um array de objetos contendo os dados cadastrados no formulário.

```
export function ListaPacientes({ pacientes }) {
```

Código-fonte 6 - Exportar a lista de pacientes
Fonte: Elaborado pelo autor (2025)

- **Estrutura visual da lista:**
  - Inicia a renderização de uma lista não ordenada (<ul>), que será preenchida dinamicamente com os pacientes cadastrados.

```
return (
  <ul>
```

Código-fonte 7 - Retornar lista não ordenada
Fonte: Elaborado pelo autor (2025)

- **Iteração com .map():**
  - Utiliza o método **.map()** do JavaScript para percorrer todos os elementos do array pacientes.
  - Para cada paciente ***p***, é retornado um item <li>.
  - ***i*** representa o índice do paciente na lista e é utilizado como key, uma exigência do React para identificar cada item da lista de forma única.

```
{pacientes.map((p, i) => (
```

Código-fonte 8 - Renderização de lista de pacientes com a função map
Fonte: Elaborado pelo autor (2025)

- **Renderização de cada item da lista:**

  Para cada paciente:

  - Exibe o nome.
  - Exibe a idade, seguida de “anos”.
  - Exibe a pressão arterial completa, no formato *sistólica* × *diastólica*, com a unidade mmHg.

```
     <li key={i}>
        {p.nome}        -        {p.idade}        anos        -
{p.pressaoSistolica}x{p.pressaoDiastolica} mmHg
     </li>
```

Código-fonte 9 - Exibição individual dos dados de cada paciente na lista
Fonte: Elaborado pelo autor (2025)

FIAP

5 INTEGRAÇÃO NO APP.JSX – ORGANIZANDO O FLUXO COMPLETO DA FASE 1

Após a criação dos componentes **FormularioPaciente** e **ListaPacientes**, chegou o momento de integrar a lógica do nosso projeto dentro do componente principal **App.jsx**, que atua como controlador da aplicação.

Neste tópico, você aprenderá a:

- Centralizar o estado global de pacientes no componente principal.
- Utilizar hooks nativos do React (**useState, useEffect**) para persistir dados no navegador com **localStorage**.
- Renderizar a interface completa unindo formulário + lista.
- Enviar funções como props para componentes filhos.

Caminho do arquivo “src/App.jsx”:



Figura 8 - Caminho do arquivo “src/App.jsx”
Fonte: Elaborado pelo autor (2025)

O arquivo App.jsx atua como componente raiz da aplicação, sendo responsável por:

- Armazenar e atualizar a lista de pacientes cadastrados.
- Carregar dados do **localStorage** quando a aplicação inicia.
- Salvar os dados automaticamente no **localStorage** sempre que houver alterações.
- Enviar para os componentes filhos (**FormularioPaciente** e **ListaPacientes**) as informações e funções necessárias via props.

Este padrão prepara você para trabalhar com aplicações de uma única página (**SPA**) com persistência local.

```
import { useState, useEffect } from 'react';
import { FormularioPaciente } from
'./components/FormularioPaciente';
import { ListaPacientes } from
'./components/ListaPacientes';
import './styles.css';

export default function App() {
  const [pacientes, setPacientes] = useState([]);
  const [jaCarregou, setJaCarregou] = useState(false);

  useEffect(() => {
    const dadosSalvos = localStorage.getItem('pacientes');
    if (dadosSalvos)
setPacientes(JSON.parse(dadosSalvos));
    setJaCarregou(true);
  }, []);

  useEffect(() => {
    // Só salva no localStorage após ter carregado os dados
iniciais
    if (jaCarregou) {
      localStorage.setItem('pacientes',
JSON.stringify(pacientes));
    }
  }, [pacientes, jaCarregou]);

  function adicionarPaciente(p) {
    setPacientes([...pacientes, p]);
  }

  return (
    <div>
      <h1>CardioIA – Batimentos de Dados</h1>
      <FormularioPaciente
onAddPaciente={adicionarPaciente} />
      <ListaPacientes pacientes={pacientes} />
    </div>
  );
}
```

Código-fonte 10 - Componente raiz com formulário, lista e persistência local
Fonte: Elaborado pelo autor (2025)

Importante:
Como estamos trabalhando com um projeto React criado do zero com Vite, o arquivo App.jsx gerado automaticamente vem com um conteúdo inicial padrão. Antes de prosseguir, **substitua todo o conteúdo atual de App.jsx pelo código que apresentamos acima**.
Essa substituição é essencial para que a lógica da Fase 1 funcione corretamente e você tenha uma base limpa para evoluir o projeto nas próximas etapas.

Explicação do código – análise técnica do componente App.jsx:

Veja o que cada parte do código faz, explicada bloco a bloco para facilitar o entendimento.

- **Importações iniciais:**
  - Importa os hooks useState e useEffect, fundamentais para controle de estado e efeitos colaterais no React.
  - Importa os componentes criados (FormularioPaciente e ListaPacientes) e o CSS global da aplicação.

```
import { useState, useEffect } from 'react';
import { FormularioPaciente } from
'./components/FormularioPaciente';
import { ListaPacientes } from './components/ListaPacientes';
import './styles.css';
```

Código-fonte 11 - Importação de hooks, componentes e estilos
Fonte: Elaborado pelo autor (2025)

- **Declaração do componente App com estados:**
  - pacientes: guarda a lista de pacientes cadastrados.
  - setPacientes: atualiza a lista.
  - jaCarregou: é um controle booleano para indicar que os dados do localStorage já foram carregados (evita sobrescrever com lista vazia na inicialização).

```
export default function App() {
  const [pacientes, setPacientes] = useState([]);
  const [jaCarregou, setJaCarregou] = useState(false);
```

Código-fonte 12 - Componente raiz com estados de pacientes e carregamento
Fonte: Elaborado pelo autor (2025)

- **useEffect – Carregando dados do localStorage:**
  - Este efeito roda **apenas uma vez** ao iniciar o app (por isso o array vazio []).
  - Busca os dados salvos com a chave "pacientes" no localStorage.
  - Se encontrar, converte a string JSON para array com JSON.parse() e atualiza o estado pacientes.
  - Depois de carregar os dados, ativa a flag jaCarregou para true.

```
useEffect(() => {
  const dadosSalvos = localStorage.getItem('pacientes');
  if (dadosSalvos) setPacientes(JSON.parse(dadosSalvos));
  setJaCarregou(true);
}, []);
```

Código-fonte 13 - Carregamento inicial dos pacientes via localStorage
Fonte: Elaborado pelo autor (2025)

- **useEffect – Salvando no localStorage:**
  - Este efeito monitora alterações no array de pacientes.
  - Só grava os dados no localStorage se jaCarregou for true, evitando sobrescrever com lista vazia logo ao abrir a aplicação.
  - Converte o array de objetos para string com JSON.stringify() antes de salvar.

```
useEffect(() => {
  if (jaCarregou) {
    localStorage.setItem('pacientes',
JSON.stringify(pacientes));
  }
}, [pacientes, jaCarregou]);
```

Código-fonte 14 - Salvando pacientes no localStorage após carregamento
Fonte: Elaborado pelo autor (2025)

- **Função para adicionar paciente:**
  - Recebe um objeto p (paciente) e o adiciona ao final do array atual.
  - A atualização de estado aciona automaticamente o useEffect, que salvará no localStorage.

```
function adicionarPaciente(p) {
  setPacientes([...pacientes, p]);
}
```

Código-fonte 15 - Função para incluir novo paciente na lista
Fonte: Elaborado pelo autor (2025)

- **Renderização da interface:**
  - Exibe o título principal da aplicação.
  - Renderiza o componente FormularioPaciente, passando a função adicionarPaciente como prop.
  - Renderiza ListaPacientes, passando a lista atual de pacientes como prop para exibição.

```
return (
  <div>
    <h1>CardioIA – Batimentos de Dados</h1>
    <FormularioPaciente onAddPaciente={adicionarPaciente} />
    <ListaPacientes pacientes={pacientes} />
  </div>
);
```

Código-fonte 16 - Estrutura da interface com título, formulário e lista de pacientes
Fonte: Elaborado pelo autor (2025)

Conclusão didática:

- O componente App é o **coração do projeto CardioIA**, integrando formulário, lista e persistência de dados.
- Utiliza dois useEffect: um para **carregar** os dados ao iniciar e outro para **salvar** ao modificar.

- A flag jaCarregou é uma prática importante para **evitar bugs de sobrescrita no localStorage**.
- Essa estrutura permite uma experiência completa de cadastro e exibição de dados, mesmo sem backend.

FIAP

6 APLICANDO ESTILO À INTERFACE – PRIMEIRA CAMADA DE APRESENTAÇÃO

Nosso projeto **CardioIA** já está funcionando: conseguimos cadastrar pacientes, exibir dados e manter tudo sincronizado com o localStorage. No entanto, do ponto de vista visual, a interface ainda está bastante crua — com elementos desalinhados, sem cores, e com uma aparência genérica de formulário HTML puro.

Mas não se preocupe. Isso é esperado!



Figura 9 - Interface do Projeto no Navegador sem estilização de CSS
Fonte: Elaborado pelo autor (2025)

Até o momento, o foco foi garantir a **estrutura lógica** e a **organização dos componentes React**, estabelecendo uma fundação sólida para a aplicação. Agora, vamos evoluir para uma nova camada do desenvolvimento frontend: a **apresentação**.

O CSS (Cascading Style Sheets) é a linguagem responsável por definir a aparência de elementos HTML: cores, tamanhos, margens, espaçamentos, fontes, animações e muito mais. A separação entre estrutura (HTML/JSX), comportamento (JavaScript) e estilo (CSS) é um dos pilares do desenvolvimento frontend moderno.

Nesta etapa, aplicaremos um conjunto de estilos globais que serão compartilhados por toda a aplicação. Para isso, criaremos uma folha de estilo centralizada, chamada styles.css, que será vinculada diretamente ao nosso App.jsx.

Essa abordagem é ideal para iniciantes por sua simplicidade, mas ao longo dos próximos capítulos você conhecerá técnicas mais avançadas como:

- Styled Components (CSS-in-JS);
- TailwindCSS (framework utilitário baseado em classes);
- CSS Modules (escopo local por componente).

Começaremos pelo essencial. Afinal, antes de voar alto, é preciso dominar o básico com firmeza.

6.1 Onde criar o arquivo de estilo

Crie o arquivo **styles.css** dentro da pasta **src**, conforme a estrutura:

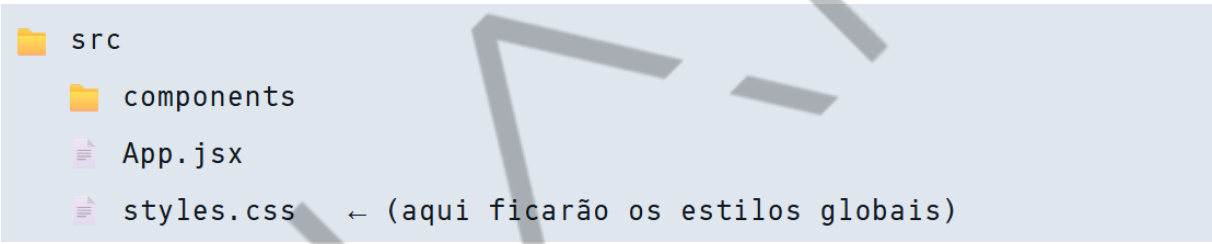


Figura 10 - Interface gráfica VsCode – Diretórios e Arquivos
Fonte: Elaborado pelo autor (2025)

Código do CSS: cole o código no seu styles.css:

```
/* Estilos Globais */
body {
  font-family: Arial, sans-serif;
  background-color: #f9f9f9;
  margin: 0;
  padding: 20px;
}
h1 {
  text-align: center;
  color: #2c3e50;
}
/* Estilo do Formulário */
form {
  display: flex;
  flex-direction: column;
  max-width: 400px;
  margin: 0 auto 30px;
  padding: 20px;
  background-color: #ffffff;
  border-radius: 8px;
```

```
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
form input {
  margin-bottom: 12px;
  padding: 10px;
  font-size: 1rem;
  border: 1px solid #ccc;
  border-radius: 4px;
}
form button {
  padding: 10px;
  background-color: #2ecc71;
  color: white;
  font-weight: bold;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
form button:hover {
  background-color: #27ae60;
}
/* Lista de Pacientes */
ul {
  list-style-type: none;
  padding: 0;
  max-width: 600px;
  margin: 0 auto;
}
ul li {
  background-color: D2D2D2;
  margin-bottom: 12px;
  padding: 15px;
  border-left: 5px solid #3498db;
  border-radius: 4px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
  color: #000000;
}

/* Campos de Pressão Arterial */
.pressao-campos {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 12px;
}
.pressao-campos input {
  flex: 1;
  margin-bottom: 0;
}
.separador-pressao {
  font-size: 1.2rem;
```

```
    font-weight: bold;
    color: #34495e;
    min-width: 20px;
    text-align: center;
}
.exemplo-pressao {
    margin: 0 0 12px 0;
    font-size: 0.9rem;
    color: #7f8c8d;
    font-style: italic;
    text-align: center;
}
```

Código-fonte 17 - Estilos globais, formulário e lista de pacientes em CSS
Fonte: Elaborado pelo autor (2025)

Assim, a aparência dos componentes **FormularioPaciente** e **ListaPacientes** ficará mais limpa, organizada e agradável para o usuário final.

6.1.1 Explicação por Blocos do styles.css

Estilos globais:

```
body {
    font-family: Arial, sans-serif;
    background-color: #f9f9f9;
    margin: 0;
    padding: 20px;
}
```

Código-fonte 18 - Configuração visual básica da página (body)
Fonte: Elaborado pelo autor (2025)

- Define a fonte padrão do sistema como Arial (com fallback para sans-serif).
- Aplica um fundo cinza claro ao corpo da página.
- Remove margens padrão e insere um espaçamento interno de 20px.

```
h1 {
    text-align: center;
    color: #2c3e50;
}
```

Código-fonte 19 - Estilo do título principal da aplicação
Fonte: Elaborado pelo autor (2025)

- Centraliza o título da página e define uma cor escura elegante para ele.

Estilo do formulário:

```
form {
  display: flex;
  flex-direction: column;
  max-width: 400px;
  margin: 0 auto 30px;
  padding: 20px;
  background-color: #ffffff;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
```

Código-fonte 20 - Layout e aparência do formulário principal
Fonte: Elaborado pelo autor (2025)

- Utiliza layout flex em coluna.
- Centraliza o formulário horizontalmente com margin: 0 auto.
- Aplica padding interno, bordas arredondadas e sombra suave para dar destaque e profundidade visual ao formulário.

```
form input {
  margin-bottom: 12px;
  padding: 10px;
  font-size: 1rem;
  border: 1px solid #ccc;
  border-radius: 4px;
}
```

Código-fonte 21 - Estilo dos campos de entrada do formulário
Fonte: Elaborado pelo autor (2025)

- Estiliza os campos do formulário com espaçamento, bordas sutis e fontes legíveis.
- Garante aparência moderna e legibilidade.

```
form button {
  padding: 10px;
  background-color: #2ecc71;
  color: white;
  font-weight: bold;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
form button:hover {
  background-color: #27ae60;
}
```

Código-fonte 22 - Estilo do botão do formulário com efeito hover
Fonte: Elaborado pelo autor (2025)

- Define um botão de cor verde vibrante com texto branco e negrito.
- Aplica efeito hover para feedback visual quando o usuário passar o mouse.

Lista de pacientes:

```
ul {
  list-style-type: none;
  padding: 0;
  max-width: 600px;
  margin: 0 auto;
}
ul li {
  background-color: #d2d2d2;
  margin-bottom: 12px;
  padding: 15px;
  border-left: 5px solid #3498db;
  border-radius: 4px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
  color: #000000;
}
```

Código-fonte 23 - Estilo da lista de pacientes e de cada item
Fonte: Elaborado pelo autor (2025)

- Remove marcadores de lista e centraliza o conteúdo da ul.
- Cada item da lista (li) recebe:
  - Cor de fundo personalizada.

- Borda lateral azul (indicando destaque).
- Cantos arredondados e sombra para separação visual.

Campos de pressão arterial:

```
.pressao-campos {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 12px;
}
```

Código-fonte 24 - Layout em linha para os campos de pressão arterial
Fonte: Elaborado pelo autor (2025)

- Organiza os dois campos de pressão em linha (lado a lado) com display: flex.
- gap: 10px insere um espaço entre os campos e o separador x.

```
.pressao-campos input {
  flex: 1;
  margin-bottom: 0;
}
```

Código-fonte 25 - Ajuste dos inputs dentro do bloco de pressão arterial
Fonte: Elaborado pelo autor (2025)

- Garante que os dois campos de pressão ocupem o mesmo espaço e fiquem alinhados.

```
.separador-pressao {
  font-size: 1.2rem;
  font-weight: bold;
  color: #34495e;
  min-width: 20px;
  text-align: center;
}
```

Código-fonte 26 - Estilo do separador entre os campos de pressão arterial
Fonte: Elaborado pelo autor (2025)

- Estiliza o x entre os campos para se destacar visualmente, com peso forte e cor escura (tom azul-petróleo).

```
.exemplo-pressao {
  margin: 0 0 12px 0;
  font-size: 0.9rem;
  color: #7f8c8d;
  font-style: italic;
  text-align: center;
}
```

Código-fonte 27 - Estilo do texto de exemplo para preenchimento da pressão arterial
Fonte: Elaborado pelo autor (2025)

- Adiciona uma legenda de exemplo para orientar o usuário sobre o formato esperado da pressão arterial.
- Apresentado de forma sutil, com cor neutra e estilo itálico.

Conclusão didática:

- Este CSS torna o formulário visualmente agradável e funcional para telas pequenas e médias.
- A separação das pressões em dois campos distintos facilita o tratamento lógico e visual dos dados.
- O botão e os campos seguem uma estética moderna e coerente com boas práticas de UX.

Onde importar o CSS:

- No topo do arquivo **App.jsx**, adicione a seguinte linha para aplicar os estilos à aplicação:

  import './styles.css'.

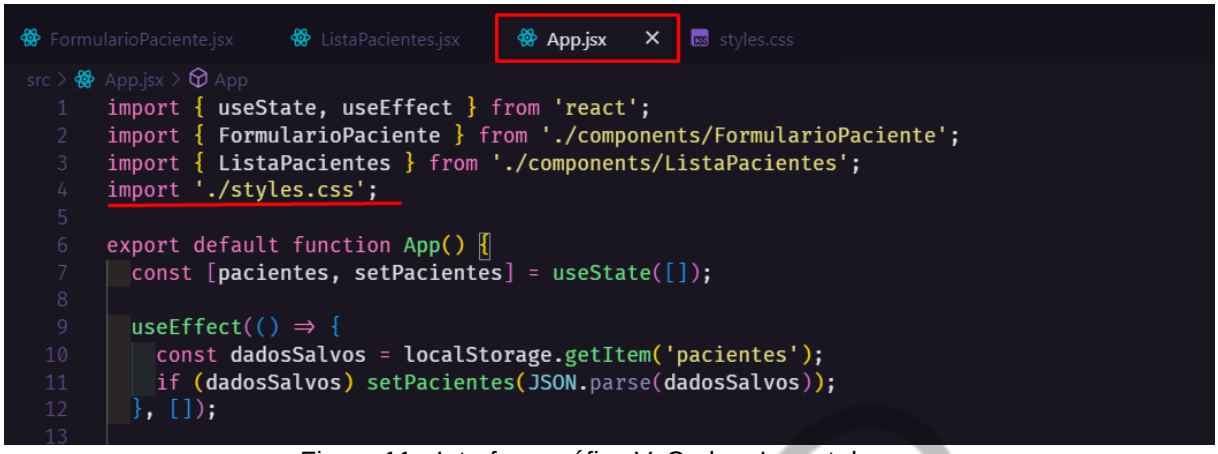


Figura 11 - Interface gráfica VsCode – Import do css
Fonte: Elaborado pelo autor (2025)

Pronto! Com isso, todos os estilos definidos no arquivo serão aplicados globalmente à interface.

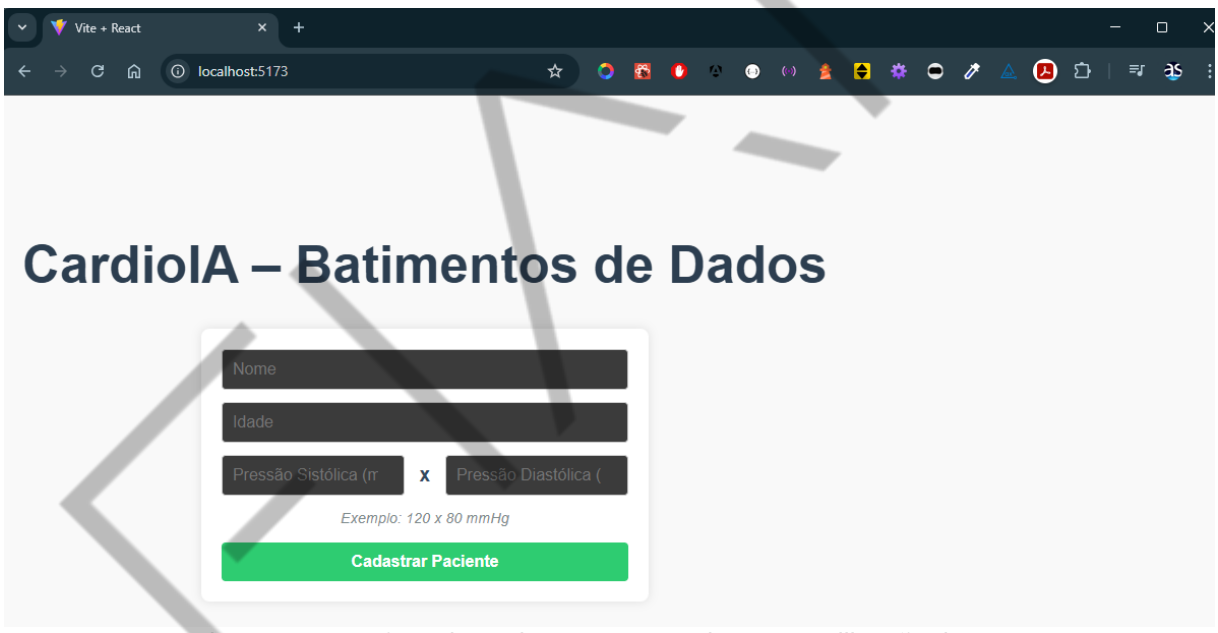


Figura 12 - Interface do Projeto no Navegador com estilização de CSS
Fonte: Elaborado pelo autor (2025)

Dica: evite editar estilos diretamente nos componentes neste momento. Essa folha centralizada facilitará a manutenção e permitirá que você foque no desenvolvimento de lógica e no comportamento da aplicação.

7 TESTANDO O FUNCIONAMENTO E VISUALIZANDO OS DADOS NO NAVEGADOR

Após cadastrar um paciente pelo formulário, repare que ele aparece automaticamente abaixo, na lista exibida na tela. Isso acontece porque os dados estão sendo armazenados e persistidos no navegador por meio do **localStorage**.

7.1 O que é o localStorage?

O localStorage é uma funcionalidade do navegador que permite armazenar dados localmente, mesmo após o fechamento da página. Ele é ideal para simular o funcionamento de um backend durante as fases iniciais de desenvolvimento Frontend.

No nosso projeto, ele está sendo utilizado para armazenar a **lista de pacientes**, garantindo que os dados não se percam quando a página for recarregada.

Mais adiante, com a introdução de um **backend**, substituiremos esse armazenamento local por um banco de dados real.

7.2 Onde está o código que grava no localStorage?

Dentro do arquivo App.jsx, observe o seguinte trecho:

```
useEffect(() => {
  localStorage.setItem('pacientes',
JSON.stringify(pacientes));
}, [pacientes]);
```

Código-fonte 28 - Atualização automática do localStorage ao mudar a lista de pacientes
Fonte: Elaborado pelo autor (2025)

Esse código é executado **toda vez que a lista de pacientes muda**. Ele converte o array de pacientes para uma string (JSON.stringify) e salva no localStorage com a chave "pacientes".

Logo acima, há outro useEffect:

```
useEffect(() => {
  const dadosSalvos = localStorage.getItem('pacientes');
  if (dadosSalvos) setPacientes(JSON.parse(dadosSalvos));
}, []);
```

Código-fonte 29 - Carregamento inicial dos pacientes salvos no localStorage
Fonte: Elaborado pelo autor (2025)

Esse trecho é executado **apenas uma vez ao abrir o app**, carregando os dados salvos e colocando-os de volta no estado React (useState). Assim, a lista reaparece mesmo após recarregar a página.

7.3 Como visualizar os dados salvos no localstorage do navegador?

Siga os passos para ver os dados salvos localmente no navegador (Google Chrome):

- **Passo 1:** abra o navegador Chrome e acesse sua aplicação (http://localhost:5173 ou equivalente).
- **Passo 2:** pressione F12 para abrir as ferramentas de desenvolvedor (DevTools).
- **Passo 3:** vá até a aba “Application”.
- **Passo 4:** no menu lateral esquerdo, procure a seção “Storage” > “Local Storage”.
- **Passo 5:** clique no endereço da aplicação (por exemplo, http://localhost:5173).
- **Passo 6:** na parte central da tela, você verá uma tabela com chaves e valores:
  - Procure a chave pacientes.
  - O valor estará no formato JSON, representando todos os pacientes cadastrados.

7.4 Por que estamos usando localStorage?

Neste momento do projeto:

- Estamos apenas com o **Frontend**, sem nenhum banco de dados real.
- Usar localStorage permite praticar a persistência de dados **sem depender de um backend**.
- É uma forma rápida, leve e funcional de manter os dados visíveis durante a navegação e recarga da página.

Em fases posteriores, substituiremos essa lógica por chamadas a APIs externas para enviar os dados a servidores e bancos reais.

7.5 Conclusão didática

- Os dados inseridos no formulário são **convertidos em objetos** e armazenados localmente no navegador.
- O useEffect é utilizado para **carregar e persistir** esses dados no React e no navegador.
- Com o DevTools, conseguimos **ver o conteúdo** salvo sem precisar sair da aplicação.
- Essa estratégia é muito útil em projetos educacionais, provas de conceito e protótipos rápidos de interfaces.

8 VISUALIZANDO O COMPORTAMENTO DO LOCALSTORAGE EM TEMPO REAL

Agora, vamos acompanhar na prática como o localStorage se comporta ao longo da interação com o formulário. Essa demonstração ajuda a consolidar o entendimento sobre **persistência de dados no Frontend** e como o estado do React se conecta ao navegador.

Cenário 1 – Antes de cadastrar qualquer paciente:

- Abra sua aplicação no navegador (por exemplo, http://localhost:5173).
- Pressione **F12** para abrir o DevTools.
- Vá até a aba **Application** e, no menu lateral esquerdo, localize:
- Storage > Local Storage > http://localhost:5173
- Na tabela que aparece, procure pela **chave pacientes**.

Resultado esperado: você verá a chave (key) pacientes com um valor igual a [], que representa um array vazio. Isso indica que **nenhum paciente foi cadastrado ainda**, mas a estrutura já está pronta para armazenar.

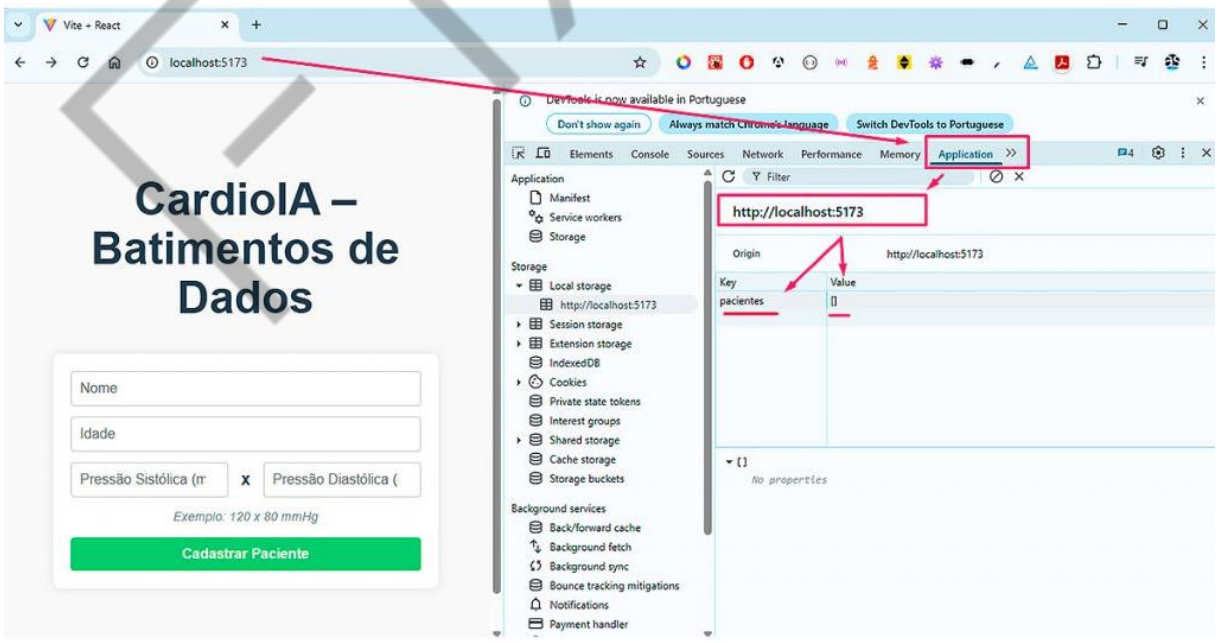


Figura 13 - Aplicação inicial sem dados preenchidos
Fonte: Elaborado pelo autor (2025)

Cenário 2 – Após cadastrar um novo paciente:

- Volte para a tela da aplicação.
- Preencha o formulário com o nome, idade e pressão do paciente.
- Clique no botão "Cadastrar Paciente".
- Agora, volte à aba **Application > Local Storage** do DevTools.

Resultado esperado: a chave (key) pacientes agora estará com um **valor em formato JSON**, representando o objeto do paciente cadastrado. Algo como:

[{"nome":"João Carlos da Silva","idade":47,"pressaoSistolica":160,"pressaoDiastolica":110}]

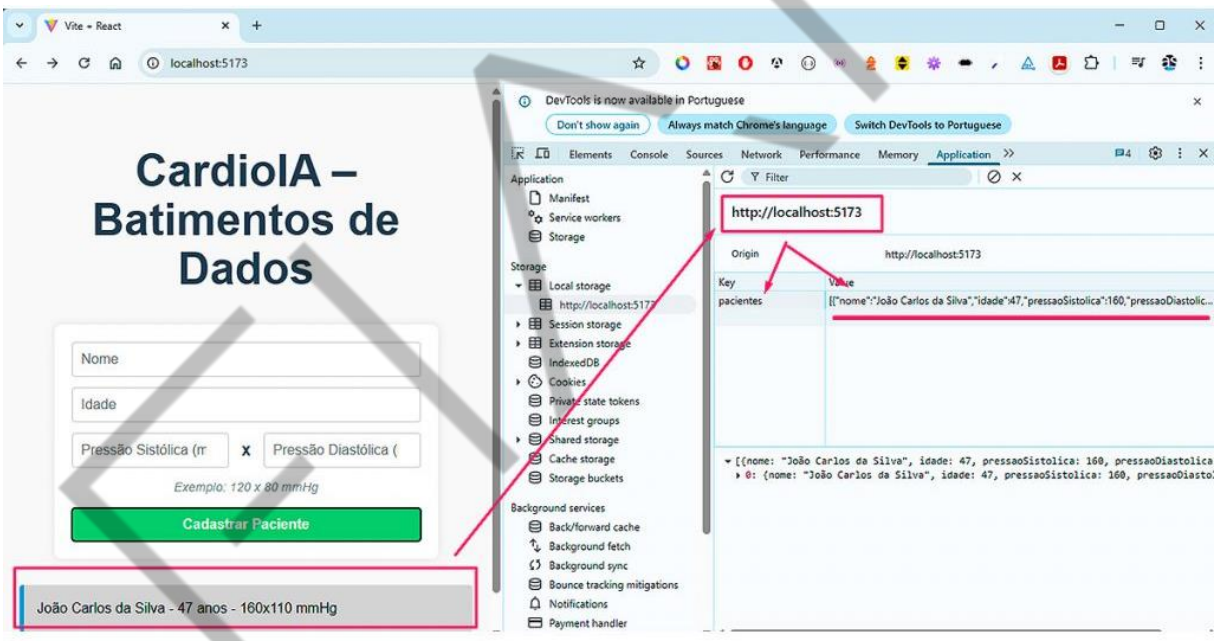


Figura 14 - Aplicação inicial com os dados preenchidos
Fonte: Elaborado pelo autor (2025)

Explicação pedagógica:

- O React, por meio do useEffect, **ouve toda alteração** no estado pacientes e salva automaticamente no localStorage.
- Isso significa que qualquer paciente adicionado (ou futuramente removido) **atualiza o localStorage imediatamente**.

- Essa sincronização é importante para garantir que os dados **não se percam** ao recarregar a página, pois o navegador irá **recarregar os dados salvos** ao iniciar a aplicação.

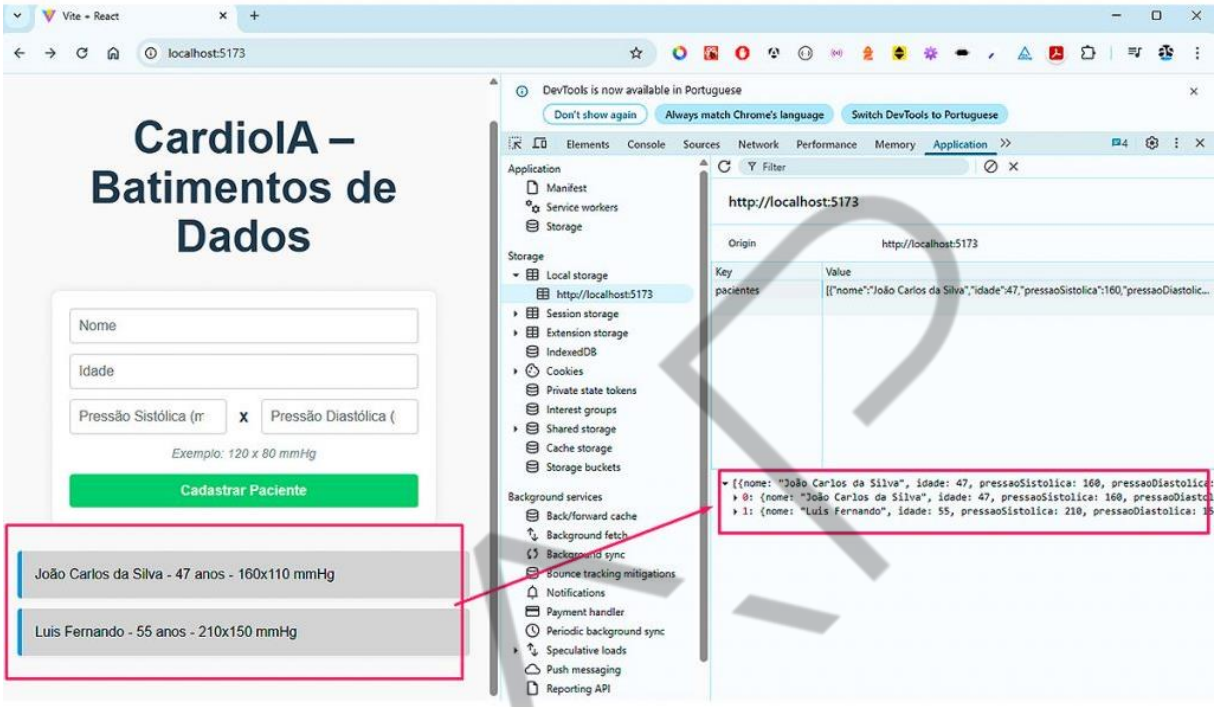


Figura 15 - Aplicação com novo dado preenchido
Fonte: Elaborado pelo autor (2025)

Conclusão didática:

- O localStorage funciona como uma simulação de banco de dados local para projetos puramente frontend.
- Ver os dados sendo atualizados **ao vivo** pelo DevTools ajuda você a enxergar como o React e o navegador estão conectados.
- Essa abordagem permite praticar lógica de estado e persistência sem depender de servidores externos.
- Mais adiante, substituiremos esse mecanismo por **chamadas reais a uma API backend** — no entanto, o conceito de “armazenar dados de forma persistente” continua sendo o mesmo.

9 VERSIONANDO O PROJETO CARDIOIA NO GITHUB

Para relembrar: no **vídeo do Capítulo 1**, você aprendeu a importância de versionar seus projetos utilizando o **GitHub** — ferramenta essencial para o controle de versões, colaboração em equipe e histórico de desenvolvimento.

A partir de agora, vamos registrar cada avanço no projeto **CardioIA**. Ao final de cada fase, será sugerido um **commit** com uma mensagem clara. Isso garantirá que você tenha um histórico completo e possa revisitar versões anteriores, se necessário.

9.1 Etapas para versionar o projeto no GitHub

Etapa 1 – Crie um novo repositório no GitHub:

- Acesse <https://github.com>;
- Clique em New repositor;
- Dê o nome cardioia-react;
- Deixe o repositório **público** ou **privado**, como preferir;
- Não marque a opção para criar README (isso evitará conflitos);
- Clique em Create repository.

Etapa 2 – Inicialize o Git na pasta do seu projeto:

- Abra o terminal na raiz do seu projeto local (onde está o package.json) e execute: git init

Etapa 3 – Adicione o repositório remoto criado no GitHub:

- Copie o link do repositório (por exemplo: https://github.com/**seuusuario**/cardioia-react.git) e execute: git remote add origin https://github.com/**seuusuario**/cardioia-react.git

Atenção: substitua **seunome** pelo seu usuário real do GitHub.

Etapa 4 – Adicione os arquivos ao controle de versão:

- git add

Etapa 5 – Faça o primeiro commit:

- git commit -m "Início do projeto CardioIA - Estrutura base com Formulário e Lista de Pacientes".

Etapa 6 – Envie seu código para o GitHub:

- git branch -M main
- git push -u origin main

Dica: adicione Commit ao final de cada fase.

Ao final de cada fase do projeto, volte ao terminal e execute os seguintes comandos para manter seu histórico de versão atualizado:

```
    git add
    git commit -m "Final da Fase 1 – Formulário e Lista com persistência em localStorage"
    git push
```

Comando de prompt 7 - Registro e envio das alterações para o repositório Git
Fonte: Elaborado pelo autor (2025)

Essa prática simples trará benefícios como:

- Facilidade para desfazer erros;
- Acompanhamento da sua evolução;
- Reutilização futura em TCCs, portfólios ou projetos reais.

10 FINALIZANDO A FASE 1 – CADASTRO E PERSISTÊNCIA DE PACIENTES

Concluímos a primeira fase do projeto **CardioIA**, onde estruturamos a aplicação, criamos os primeiros componentes com React, implementamos o cadastro de pacientes e aplicamos uma camada de estilização visual. Além disso, garantimos que os dados fossem salvos de forma persistente no navegador por meio do localStorage, mesmo sem um backend real.

Essa estrutura inicial é fundamental para consolidar os conceitos de estado, componentes controlados e persistência local no Frontend.

FIAP

REFERÊNCIAS

PONTES, Guilherme. **Progressive web apps:** construa aplicações progressivas com React. 1. ed. São Paulo: Casa do Código, 2018. 441 p.

FIAP